codigofacilito

Clase 11 · Bootcamp AI Engineer

Multiagentes con LangGraph

Orquestación stateful mediante grafos

Instructor: Ramsés Camas

¿Qué veremos hoy?

01 Recapitulación: De pipelines a agentes a ReAct

02 ¿Por qué necesitamos sistemas multiagente?

03 Patrones de orquestación multiagente

04 LangGraph: grafos como arquitectura

05 Arquitectura Supervisor–Workers

06 Hand-offs, fallbacks y bucles de calidad

07 Proyecto: agents/multi_agent_graph.py

Recapitulación: ¿Qué es un agente?

De pipelines predefinidos a decisiones dinámicas

Componentes de un agente

- Un LLM como cerebro decisor
- Un conjunto de herramientas (tools)
- Un estado que mantiene el contexto
- Un loop: pensar → actuar → observar

Pipeline vs Agente

Pipeline:

A → B → C siempre en ese orden

Agente:

"¿Necesito buscar? ¿Calcular?
¿Ya tengo la respuesta?"

Agente simple vs Agente ReAct

La importancia del razonamiento explícito

Agente simple (tool-calling)

El LLM decide si llamar una herramienta o responder directamente.

El "pensamiento" está implícito en la generación del modelo.

Loop: pensar → (actuar → observar)* → responder

Agente ReAct (Reason + Act)

Patrón explícito en tres fases:

Thought: razona qué necesita hacer
Action: ejecuta herramienta
Observation: incorpora resultado

Decisiones más confiables y trazas legibles

Recap — Nuestro agente ReAct (Clase 10)

```
from langgraph.graph import StateGraph, START, END, MessagesState

def llm_call(state: MessagesState):
    """El LLM razona y decide si usar una herramienta"""
    return {"messages": [model_with_tools.invoke(state["messages"])]}

def tool_node(state: MessagesState):
    """Ejecuta la herramienta seleccionada"""
    result = []
    for tool_call in state["messages"][-1].tool_calls:
        tool = tools_by_name[tool_call["name"]]
        observation = tool.invoke(tool_call["args"])
        result.append(ToolMessage(content=observation,
                                  tool_call_id=tool_call["id"]))
    return {"messages": result}

def should_continue(state: MessagesState):
    if state["messages"][-1].tool_calls:
        return "tool_node"    # Actuar
    return END                # Responder al usuario
```

Un solo agente, un solo LLM, un conjunto de herramientas. Este es nuestro punto de partida.

¿Por qué un solo agente no es suficiente?

Tres problemas fundamentales al escalar

1

Sobrecarga de contexto

20 herramientas = malas decisiones. El LLM se confunde con demasiadas opciones e instrucciones.

2

Falta de especialización

Buscar, analizar, formatear y validar son 4 trabajos distintos. Mezclarlos degrada la calidad.

3

Escalabilidad del desarrollo

Todo en un agente = cualquier cambio puede romper todo. Imposible trabajo en equipo.

Analogía: Un empleado que es recepcionista, contador, desarrollador y gerente al mismo tiempo.

Sistemas multiagente: múltiples cerebros coordinados

Múltiples agentes independientes, cada uno con su propio LLM, prompt y herramientas

Gestión de contexto

Cada agente recibe solo el conocimiento relevante para su tarea. No saturamos la ventana de contexto.

Desarrollo distribuido

Equipos construyen y mantienen agentes especializados de forma independiente.

Paralelización

Agentes especializados ejecutan subtareas simultáneamente, reduciendo latencia total.

Cada agente es un nodo en un grafo. Sus conexiones son aristas. Se comunican actualizando un estado compartido.

Un agente como nodo en un grafo

```
from langgraph.graph import StateGraph, START, END
from typing_extensions import TypedDict, Annotated
import operator

# Estado compartido entre todos los agentes
class MultiAgentState(TypedDict):
    messages: Annotated[list, operator.add]  # Historial compartido
    plan: str                                 # Plan del planner
    search_results: str                       # Resultados del retriever
    draft: str                                # Borrador del executor
    quality_score: float                      # Score del verifier
    iteration: int                            # Iteraciones

# Cada agente es una función que lee y escribe al estado
def planner_agent(state: MultiAgentState):
    """Analiza la tarea y genera un plan"""
    response = planner_llm.invoke(
        f"Genera un plan para: {state['messages'][-1].content}"
    )
    return {"plan": response.content}
```

Cada agente recibe el estado y retorna actualizaciones parciales.

Cuatro patrones de orquestación

Diferentes formas de coordinar múltiples agentes

1

Supervisor–Workers

Un supervisor descompone tareas, delega a workers, sintetiza resultados.

2

Handoffs

Agentes se transfieren control entre sí. Dinámico, cada agente habla con el usuario.

3

Router

Clasifica input y dirige al especialista. Como un switch/case inteligente.

4

Subagentes

Agente principal invoca subagentes como herramientas. Fuerte aislamiento de contexto.

¿Cuándo usar cada patrón?

Patrón	Mejor para	Trade-off	Ejemplo
Supervisor–Workers	Tareas descomponibles, paralelización	Más llamadas LLM	Reporte con secciones
Handoffs	Conversación multi-turno	No paralelizable	Soporte escalonado
Router	Clasificar y dirigir input	Stateless	Ventas/soporte/fact.
Subagentes	Aislamiento de contexto	Mayor overhead	Múltiples KBs

Para DocOps Agent usaremos Supervisor–Workers: planner, retriever, executor y verifier.

Ejemplo — Router que clasifica y delega

```python
from pydantic import BaseModel, Field
from typing import Literal

class Route(BaseModel):
    target: Literal["search", "analysis", "summary"] = Field(
        description="Agente al que enviar la consulta")

router_llm = llm.with_structured_output(Route)

def router_node(state: MultiAgentState):
    """Clasifica el input y decide qué agente debe actuar"""
    decision = router_llm.invoke(
        f"Clasifica esta consulta: {state['messages'][-1].content}")
    return {"next_agent": decision.target}

def route_decision(state: MultiAgentState):
    return state["next_agent"]
```

```python
graph = StateGraph(MultiAgentState)
graph.add_node("router", router_node)
graph.add_node("search_agent", search_agent)
graph.add_node("analysis_agent",
analysis_agent)
graph.add_edge(START, "router")
graph.add_conditional_edges("router",
route_decision, {
    "search": "search_agent",
    "analysis": "analysis_agent",
    "summary": "summary_agent"})
```

with_structured_output fuerza al LLM a retornar un objeto Route validado (contratos de Clase 4).

Supervisor–Workers

El patrón que usaremos para DocOps Agent

Supervisor–Workers: cómo funciona

Delegación, especialización y síntesis

El Supervisor

- Recibe la tarea del usuario
- Genera plan de ejecución
- Delega subtareas a workers
- Recibe resultados de cada worker
- Sintetiza la respuesta final

Los Workers

- System prompt optimizado para su tarea
- Acceso solo a herramientas necesarias
- Reciben subtarea específica
- Retornan resultado al estado
- No conocen a los otros workers

DocOps Agent — Arquitectura de 4 agentes

Flujo: Usuario → Planner → Retriever → Executor → Verifier → Respuesta

Planner

Analiza consulta, genera plan estructurado con pasos a seguir

→

Retriever

Busca en vector store, aplica búsqueda híbrida y reranking

→

Executor

Recibe plan + documentos, genera respuesta fundamentada

→

Verifier

Evalúa calidad, verifica faithfulness, asigna quality_score

Bucle de calidad: Si quality_score < 0.8, el Verifier reenvía al Executor con feedback. Máximo 3 iteraciones.

Implementando los workers del DocOps Agent

```python
from langchain.messages import SystemMessage, HumanMessage

def planner_agent(state: MultiAgentState):
    """Planner: genera un plan de acción"""
    response = llm.invoke([
        SystemMessage(content="""Eres un planificador experto.
        Analiza la consulta y genera un plan con pasos claros."""),
        HumanMessage(content=state["messages"][-1].content)
    ])
    return {"plan": response.content, "iteration": 0}

def retriever_agent(state: MultiAgentState):
    """Retriever: busca info relevante usando RAG"""
    docs = vectorstore.similarity_search(state["plan"], k=5)
    context = "\n".join([d.page_content for d in docs])
    return {"search_results": context}
```

```python
def executor_agent(state: MultiAgentState):
    """Executor: genera respuesta con plan +
contexto"""
    response = llm.invoke([
        SystemMessage(content="Genera una
respuesta precisa."),
        HumanMessage(content=f"Plan:
{state['plan']}\n"
                     f"Contexto:
{state['search_results']}")
    ])
    return {"draft": response.content}
```

Cada worker es una función simple. La magia está en cómo los conectamos.

El estado como canal de comunicación

Los agentes NO se comunican directamente — leen y escriben a un estado compartido

¿Cómo funciona?

1. El estado es un TypedDict con todos los campos
2. Cada agente recibe el estado completo como input
3. Retorna diccionario con claves a actualizar
4. LangGraph fusiona las actualizaciones

¿Por qué?

- Sin ruteo peer-to-peer
- Cada agente lee solo lo que necesita
- Memoria de trabajo compartida
- Debugging: inspeccionar estado en cualquier punto

Para listas (como messages) usamos Annotated con operator.add para agregar en vez de reemplazar.

Controlando el flujo con aristas

Tres tipos de aristas en LangGraph

add_edge (directa)

Flujo fijo: siempre va de A a B.
"Después de planificar, siempre busca."

add_conditional_edges

Una función evalúa el estado y decide.
"Si score bajo → executor. Si alto → END."

Send API (dinámico)

Crea workers bajo demanda.
"Para cada sección, crea un worker paralelo."

El bucle de calidad

La arista condicional después del Verifier transforma un pipeline lineal en un sistema con auto-corrección. El sistema itera sobre su salida hasta alcanzar el estándar de calidad, con límite de iteraciones para evitar loops infinitos.

Verifier con bucle de retroalimentación

```
from pydantic import BaseModel, Field
from typing import Literal

class QualityCheck(BaseModel):
    score: float = Field(description="Score de calidad 0.0 a 1.0")
    feedback: str = Field(description="Retroalimentación si score bajo")
    decision: Literal["accept", "revise"]

quality_checker = llm.with_structured_output(QualityCheck)

def verifier_agent(state: MultiAgentState):
    """Verifier: evalúa calidad y decide si aceptar o revisar"""
    check = quality_checker.invoke(
        f"Evalúa esta respuesta:\n{state['draft']}\n\n"
        f"Basada en estos documentos:\n{state['search_results']}")
    return {
        "quality_score": check.score,
        "feedback": check.feedback,
        "iteration": state["iteration"] + 1}
```

```
def should_revise(state: MultiAgentState):
    """Arista condicional: ¿revisar o
aceptar?"""
    if state["quality_score"] >= 0.8:
        return "accept"
    if state["iteration"] >= 3:       #
Máximo 3 iteraciones
        return "accept"
    return "revise"
```

El verifier usa structured output para formato consistente. El límite de iteraciones previene loops infinitos.

Hand-offs: transferencia de control

Agentes que se pasan la responsabilidad directamente, sin supervisor central

¿Cómo funciona en LangGraph?

1. Agente ejecuta tool call que actualiza estado
2. Arista condicional redirige al siguiente agente
3. Nuevo agente toma control con su propio prompt
4. Usuario experimenta conversación continua

Ejemplo: Soporte

Nivel 1 → consultas generales

Nivel 2 → casos complejos

Humano → escalamiento (HITL, Clase 12)

Trade-off: Los hand-offs son secuenciales — no puedes paralelizar. Ideales para interacción directa con el usuario.

Fallbacks: qué hacer cuando algo falla

En producción, los agentes fallan. Un sistema robusto necesita fallbacks en cada nivel.

Nivel 1

Reintentos con backoff

Llamada LLM falla → reintentar con delay exponencial. Máximo 3 intentos.

Nivel 2

Agente alternativo

Agente principal falla → redirigir a uno con modelo más pequeño.

Nivel 3

Respuesta degradada

Todos fallan → respuesta honesta con resultado parcial obtenido.

Nivel 4

Escalada a humano

Tarea crítica → activar HITL que implementaremos en Clase 12.

Principio: Nunca dejar al usuario sin respuesta. Fallo silencioso es peor que "no sé".

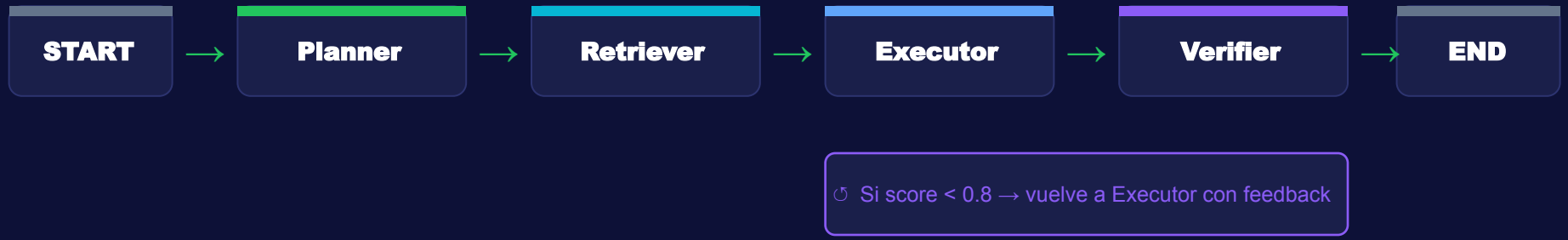
Fallback con try/except y agente de respaldo

```
def executor_with_fallback(state: MultiAgentState):
    """Executor con fallback a modelo más pequeño"""
    try:
        # Intento principal con modelo fuerte
        response = primary_llm.invoke([
            SystemMessage(content="Genera una respuesta completa."),
            HumanMessage(content=f"Plan: {state['plan']}\n"
                         f"Contexto: {state['search_results']}")])
        return {"draft": response.content}
    except Exception as e:
        # Fallback a modelo más económico
        try:
            response = fallback_llm.invoke([
                SystemMessage(content="Genera una respuesta concisa."),
                HumanMessage(content=f"Contexto: {state['search_results']}")])
            return {"draft": f"[Respuesta de respaldo]\n"
                            f"{response.content}"}
        except Exception as e2:
            # Último recurso: respuesta degradada
            return {"draft": f"No pude generar respuesta. "
                    f"Documentos encontrados:\n"
                    f"{state['search_results'][:500]}",
                    "quality_score": 0.3}
```

En producción, estos errores se registran en el sistema de observabilidad (Clase 16).

Ensamblando el grafo multiagente

Todas las piezas conectadas con aristas directas y condicionales



El sistema resultante:

1. **Planifica** antes de actuar (no busca a ciegas)
2. **Busca** información de forma focalizada
3. **Genera** respuesta fundamentada en documentos
4. **Verifica** su propia calidad y se auto-corrige si es necesario (máx 3x)

Pipeline vs Agente vs ReAct vs Multiagente

	Flujo	Decisiones	Trazabilidad	Mejor para
Pipeline	A → B → C fijo	Ninguna	Total	Tareas repetitivas
Agente simple	LLM decide tools	Implícitas	Baja	Tools claras
ReAct	Think→Act→Obs	Explícitas	Alta	Razonamiento visible
Multiagente	Grafos + aristas	Distribuidas	Por agente	Tareas complejas

Cada nivel añade complejidad pero también capacidad. Elegir el nivel correcto es una decisión de ingeniería.

agents/multi_agent_graph.py — Grafo completo

```python
"""DocOps Agent — Sistema multiagente con LangGraph"""
from langgraph.graph import StateGraph, START, END
from typing_extensions import TypedDict, Annotated
from langchain.messages import AnyMessage
import operator

class DocOpsState(TypedDict):
    messages: Annotated[list[AnyMessage], operator.add]
    plan: str
    search_results: str
    draft: str
    feedback: str
    quality_score: float
    iteration: int
```

```python
workflow = StateGraph(DocOpsState)

workflow.add_node("planner", planner_agent)
workflow.add_node("retriever", retriever_agent)
workflow.add_node("executor", executor_with_fallback)
workflow.add_node("verifier", verifier_agent)

workflow.add_edge(START, "planner")
workflow.add_edge("planner", "retriever")
workflow.add_edge("retriever", "executor")
workflow.add_edge("executor", "verifier")

# Bucle de calidad
workflow.add_conditional_edges("verifier",
should_revise, {
    "accept": END,
    "revise": "executor",   # Vuelve con feedback
})

docops_agent = workflow.compile()
```

Tip: docops_agent.get_graph(xray=True).draw_mermaid_png() genera un diagrama visual.

Recapitulación y siguiente paso

Lo que construimos hoy

- Progresión pipeline → agente → ReAct → multiagente
- 4 patrones de orquestación
- Sistema supervisor–workers con 4 roles
- Bucle de calidad con auto-corrección
- Fallbacks para resiliencia

Clase 12: Memoria + HITL

- Checkpointing del grafo
- Memoria de conversación persistente
- Gates de aprobación humana
- Verifier escala a humano

Artefacto: `agents/multi_agent_graph.py`

Tarea

Añadir un 5to agente "formatter" entre verifier y END que formatee la respuesta en Markdown con citas a documentos fuente.